Logistická centrála

Zadání

Do logistické centrály přijíždí dodávky v intervalech exponenciálně se středem 2 hodiny. Dodávka přistupuje k centrále přes jednu nakládací/vykládací rampu. Dodávka přiveze 100 balíků ke zpracování. Dobu vyložení balíků zanedbáváme. Zpracováním přivezených balíků se zabývá jeden pracovník. Zpracování každého jednotlivého balíku trvá exponenciálně se středem 1 minuta. Zpracované balíky jsou umístěny v centrále. Kapacitu centrály pro uložení přivezených nebo zpracovaných balíků nezkoumáme. Polovina dodávek po vyložení balíků nakládá zpracované balíky až po svou kapacitu 100 balíků, druhá polovina po vyložení okamžitě opouští systém. Naložení každého jednotlivého balíku trvá exponenciálně se středem 1 minuta. Po plném naložení dodávka odjíždí a další její chování nezkoumáme.

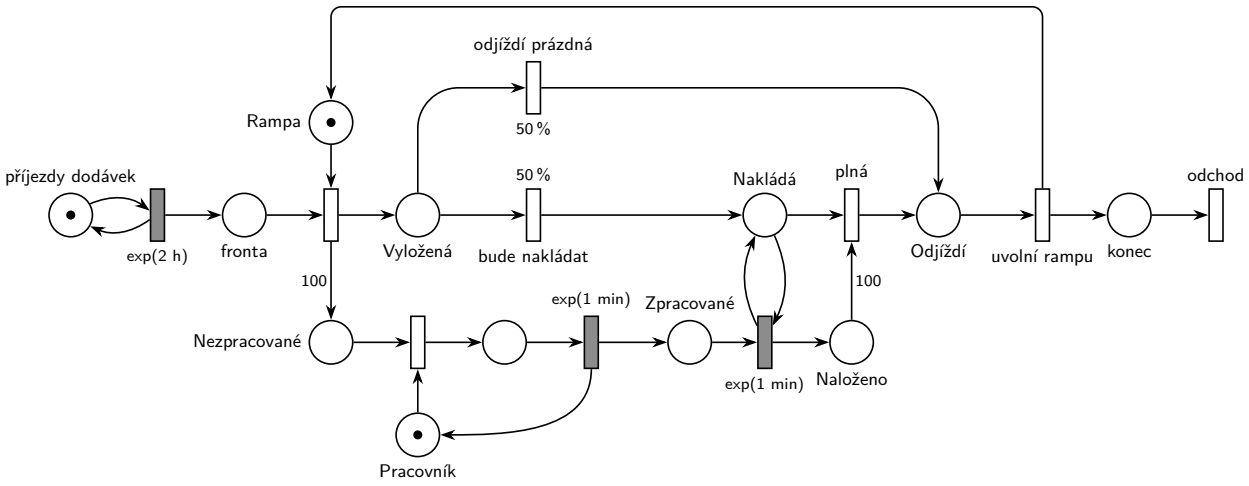
Poznámka: Pracovník zpracovávající přivezené balíky je nezávislý proces.

Podobné úlohy: Výletní loď (čítač do plna), Přepravní firma, Chmelová brigáda (stejný rok).

Nápověda

- **Příjezdy dodávek** modeluje místo s jednou značkou ve smyčce s `exp(2 h)`. Nová dodávka čeká ve *frontě* na rampu.
- **Rampa** je zařízení s jednou značkou. Dodávka ji drží od vyložení až do odjezdu, i během nakládání.
- **Vyložení trvá nulový čas**, proto je to jediný okamžitý přechod: obsadí rampu a hranou s váhou 100 vloží 100 značek do *Nezpracované*. Dodávka na zpracování svých balíků nečeká.
- **Pracovník je nezávislý proces:** vlastní zařízení *Pracovník*, které bere balíky z *Nezpracované* po jednom (`exp(1 min)`, uvolnění je sloučené s časovaným přechodem) a dává je do *Zpracované*. Do centrály se dostávají balíky ze všech dodávek dohromady.
- **50 % / 50 %** hned po vyložení: *odjíždí prázdná*, nebo *bude nakládat*. Nakládající dodávka je v místě *Nakládá*. Časovaný `exp(1 min)` ve smyčce s *Nakládá* bere vždy jeden zpracovaný balík a přidá ho do *Naloženo*, takže se nakládá po jednom. Když zpracované balíky dojdou, dodávka na rampě čeká.
- **Plná dodávka:** okamžitý přechod *plná* s hranou váhy 100 z *Naloženo*. Má přednost před dalším naložením (okamžitý přechod před časovaným), takže se 101. balík nenaloží. Naložené balíky odjíždějí s dodávkou, v síti zaniknou v přechodu *plná*.
- Obě větve se sejdou v místě *Odjíždí*. Přechod *uvolní rampu* vrátí značku rampě a dodávka odchází přechodem *odchod*. Kdo nakládá, zadání neříká. Pracovník to podle poznámky není, nakládání proto žádné další zařízení nebere.
- **Zásoba v centrále roste bez omezení:** přiváží každá dodávka, odváží jen polovina. Zadání to dovoluje („Kapacitu centrály … nezkoumáme“).

Petriho síť



Časté chyby

- Dodávka čeká na rampě, než se její balíky zpracují. Vyložení je okamžité a pracovník pracuje nezávisle.
- Pracovník, kterému se značka nevrací.
- Nakládání bez místa *Nakládá* ve smyčce: časovaný přechod jen se vstupem *Zpracované* by nakládal všechny balíky najednou (každá značka má vlastní odpočet, slajd 138).
- Odjezd bez hrany váhy 100, nebo bez návratu značky *Rampě*.
- Rozhodnutí 50 % / 50 % až po zpracování nebo s parametrem na hraně místo na přechodu.

SIMLIB

Model je izomorfní k Petriho síti výše: každé místo, přechod a značka mají svůj protějšek v kódu.

```
// Logistická centrála – model v SIMLIB/C++
#define I_REALLY_KNOW_HOW_TO_USE_WAITUNTIL  // SIMLIB zpřístupní WaitUntil jen s
 ↪  tímto makrem
#include "simlib.h"

Facility Rampa("Rampa");                // místo Rampa (1 značka)
Facility Pracovnik("Pracovník");        // místo Pracovník (1 značka)
int zpracovane = 0;                     // místo Zpracované (počet značek)
int prazdne = 0, plne = 0;              // kolik dodávek odjelo prázdných / plných
Histogram Doba("Doba dodávky v centrále [min]", 0, 30, 20);

class Balik : public Process {          // značka v místě Nezpracované
  void Behavior() {
    Seize(Pracovnik);                  // fronta Nezpracované + obsazení pracovníka
    Wait(Exponential(1));              // časovaný přechod exp(1 min) ...
    Release(Pracovnik);                // ... který vrací značku pracovníkovi
    zpracovane++;                      // značka do místa Zpracované
  }
};

class Dodavka : public Process {        // transakce = značka dodávky
  void Behavior() {
    double prijezd = Time;
    Seize(Rampa);                      // fronta + vyložení (obsadí rampu)
    for (int i = 0; i < 100; i++)      // hrana s váhou 100 do Nezpracované
      (new Balik)->Activate();
    if (Random() < 0.5) {              // 50 % bude nakládat
      int nalozeno = 0;                // místo Naloženo
```

```
      while (nalozeno < 100) {          // přechod plná (váha 100) předběhne další
       ↪  naložení
        WaitUntil(zpracovane > 0);      // Nakládá ⇄ exp(1 min) čeká na zpracovaný
 ↪  balík
        Wait(Exponential(1));           // naložení jednoho balíku
        zpracovane--;                   // balík ze Zpracované ...
        nalozeno++;                     // ... do Naloženo
      }
      plne++;
    } else {
      prazdne++;                        // 50 % odjíždí prázdná
    }
    Release(Rampa);                     // místo Odjíždí, přechod uvolní rampu
    Doba(Time - prijezd);
  }                                     // konec Behavior = přechod odchod
};

class Generator : public Event {        // místo „příjezdy dodávek“ ve smyčce s
 ↪  exp(2 h)
  void Behavior() {
    (new Dodavka)->Activate();          // nová značka v místě fronta
    Activate(Time + Exponential(2 * 60));
  }
};

int main() {
  Init(0, 60 * 24 * 365);               // modelový čas v minutách: jeden rok
  (new Generator)->Activate();
  Run();
  Rampa.Output();
  Pracovnik.Output();
  Doba.Output();
  Print("Dodávky: %d prázdných, %d plných; zásoba zpracovaných balíků: %d\n",
        prazdne, plne, zpracovane);
}
```

Petriho síť	SIMLIB
příjezdy dodávek ⇄ `exp(2 h)`	událost `Generator`, která se znovu plánuje za `Exponential(2 * 60)`
značka dodávky	proces `Dodavka`
fronta + vyložení (obsadí *Rampu*)	`Seize(Rampa)`
hrana s váhou 100 do *Nezpracované*	100 nových procesů `Balik`
Nezpracované → obsazení → `exp(1 min)` (vrací *Pracovníka*)	`Seize(Pracovnik); Wait(Exponential(1)); Release(Pracovnik);`
místo *Zpracované*	`int zpracovane`
`50 %` / `50 %`	`if (Random() < 0.5)`
Nakládá ⇄ `exp(1 min)` (bere zpracovaný balík)	`WaitUntil(zpracovane > 0); Wait(Exponential(1)); zpracovane--;`
Naloženo, přechod *plná* (váha 100)	`int nalozeno`, konec cyklu `while (nalozeno < 100)`
Odjíždí → *uvolní rampu*	`Release(Rampa)`
přechod *odchod*	konec metody `Behavior()`

Značka zpracovaného balíku zůstává v síti během odpočtu naložení v místě *Zpracované* (slajd 137). Proto kód odečte balík až po `Wait`. Jiný odběratel zpracovaných balíků ale není, takže na výsledku to nic nemění.

Kontrola simulací. Petriho síť: *Rampa* + *Vyložená* + *Nakládá* + *Odjíždí* = 1, *Pracovník* + zpracovávaný balík = 1, *Naloženo* ≤ 100. Zásoba *Zpracované* roste (asi o 50 balíků na dodávku). Fronta *Nezpracované* je stabilní (pracovník je vytížen asi na 83 %), ale po dávkách po 100 občas naroste

na stovky kusů (`make check`). SIMLIB model: v jednom běhu (1 rok, 4317 dodávek, polovina plných) byl pracovník vytížen na 82 % a rampa na 41 %; teoreticky je to 83 % (100 balíků × 1 min za 120 min) a 42 % (polovina dodávek drží rampu asi 100 min nakládáním). Fronta před pracovníkem dosáhla až 1831 balíků a zásoba zpracovaných balíků na konci roku byla asi 216 000 (`make simlib`).

Původ a zdroje

Původ: půlsemestrální písemka 2010/11 (bodování neuvedeno).

Původní znění:

> Do logistické centrály přijíždí dodávky v intervalech daných exponenciálním rozložením se středem 2 hodiny. Dodávka přistupuje k centrále přes jednu nakládací/vykládací rampu. Dodávka přiveze 100 balíků ke zpracování. Dobu vyložení balíků zanedbáváme. Zpracováním přivezených balíků se zabývá jeden pracovník. Doba zpracování každého jednotlivého balíku se řídí exponenciálním rozložením se středem 1 minuta. Zpracované balíky jsou umístěny v centrále. Kapacitu centrály pro uložení přivezených nebo zpracovaných balíků nezkoumáme. Polovina dodávek po vyložení balíků nakládá zpracované balíky až po svou kapacitu 100 balíků, druhá polovina po vyložení okamžitě opouští systém. Doba naložení každého jednotlivého balíku se řídí exponenciálním rozložením se středem 1 minuta. Po plném naložení dodávka odjíždí a další její chování nezkoumáme.
>
> Poznámka: Pracovník zpracovávající přivezené balíky je nezávislý proces.

Zdroje:

- `Fitcheats/Půlsemestrálky/Zadání z minulých let/2010-11.pdf`, str. 1 (zadání, úloha 2)
- `Fitcheats/Skripta/FIT - Studentská skripta 2015-16.pdf`, str. 44 (zadání a ručně kreslené studentské řešení s komentáři)

Existující řešení:

- **Studentská skripta, str. 44 (ručně kreslené):** „PRICHÁDZA DODÁVKA“ `exp(2h)` → přechod s „RAMPA“ → váha 100 do „BALÍKY NA VYLOŽENIE“ → `exp(1min)` s „PRACOVNÍK“ → „VYLOŽENÉ BALÍKY“ a „CENTRÁLA“ → váha 100 → `50%`: „PO VYLOŽENÍ ODCHOD“ / `50%`: „NAKLÁDÁ“ → `exp(1min)` (bere z centrály) → „NALOŽENÉ“ → váha 100 → „NALOŽENÁ ODCHÁDZA“. Obě odjíždějící větve vracejí rampu. Chyby:
  - pracovník se po zpracování balíku nevrací (komentář „Ten pracovník se nevrací?“). Zpracuje se tak jediný balík;
  - dodávka v síti zmizí a dál ji zastupuje 100 značek „vyložené balíky“. Rampu tedy drží, dokud pracovník nezpracuje všech 100 balíků. To odporuje „dobu vyložení zanedbáváme“ i poznámce, že pracovník je nezávislý proces. Navíc se tam míchají balíky různých dodávek;
  - chybí okamžité obsazení pracovníka (komentář pod řešením). Časovaný přechod se vstupem z místa zařízení by ale fungoval také.
- Diskuse pod řešením („Neměli by tam být odkonce 2 pracovníci, zvlášť na zpracování a nakládání?“) skončila správně: „v zadani je ze pracovnik se zabyva zpracovanim, nic vic“.

Reference

- Jak síť funguje
- Notace a vzory

- Studijní plán
- Seznam úloh